



**Politechnika
Śląska**

PRACA MAGISTERSKA

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych
obiektów 3D

Piotr MARCOL

Nr albumu: 300463

Kierunek: Informatyka

Specjalność: Oprogramowanie Systemowe

PROWADZĄCY PRACĘ

Dr hab. inż., prof. PŚ Michał Maćkowski

KATEDRA Systemów Rozproszonych i Urządzeń Informatyki

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2026

DRAFT

Tytuł pracy

System przeznaczony dla osób niewidomych do rozpoznawania dotykanych obiektów 3D

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

Thesis title

A system designed for the blind to recognize touched 3D objects

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

DRAFT

Spis treści

1	Wstęp	1
1.1	Motywacja podjęcia tematu	1
1.2	Cel i zakres pracy	1
1.3	Główne założenia i ograniczenia	1
1.4	Wkład autora	2
1.5	Struktura pracy	2
2	Analiza problemu i przegląd literatury	3
2.1	Perspektywa osób niewidomych w rozpoznawaniu obiektów przestrzennych	3
2.2	Technologie wspomagające osoby z niepełnosprawnością wzroku	6
2.2.1	Rozwój technologii asystujących	6
2.2.2	Standardy i regulacje prawne dotyczące dostępności	7
2.2.3	Współczesne systemy wspomagające osoby niewidome	8
2.3	Problem rozpoznawania obiektów 3D w interakcji dotykowej	10
2.4	Metody detekcji obiektów w obrazach	11
2.4.1	Sformułowanie problemu detekcji obiektów	12
2.4.2	Klasyczne metody wizji komputerowej	13
2.4.3	Detektory dwuetapowe	13
2.4.4	Detektory jednoetapowe	14
2.4.5	Detektory oparte na architekturze transformera	15
2.4.6	Porównanie metod detekcji	15
2.5	Rodzina modeli YOLO	17
2.5.1	Architektura modeli YOLO	17
2.5.2	Warianty ze względu na realizowane zadanie	20
2.5.3	Warianty ze względu na skalę modelu	21
2.5.4	Dobór wariantów modelu	22
2.6	Wykorzystanie danych RGB, głębi i LiDAR w rozpoznawaniu obiektów . .	22
2.7	Metryki oceny detekcji obiektów	23
2.8	Podsumowanie analizy tematu	25

3	Koncepcja systemu rozpoznawania dotykanych obiektów 3D	27
3.1	Założenia funkcjonalne i niefunkcjonalne	27
3.2	Architektura systemu	27
3.3	Dobór klas rozpoznawanych obiektów	27
3.4	Uzasadnienie wyboru metody detekcji	27
3.5	Przepływ danych w systemie	28
3.6	Ograniczenia przyjętej koncepcji	28
4	Implementacja systemu	29
4.1	Środowisko programistyczne i wykorzystane narzędzia	29
4.2	Moduł akwizycji danych obrazowych	29
4.3	Moduł komunikacji klient-serwer	29
4.4	Moduł detekcji obiektów	29
4.5	Moduł informacji zwrotnej dla użytkownika	29
4.6	Organizacja kodu i plików projektu	30
5	Zbiór danych i metodyka badań	31
5.1	Charakterystyka zbioru danych	31
5.2	Proces pozyskiwania i oznaczania danych	31
5.3	Podział danych na zbiory treningowy, walidacyjny i testowy	31
5.4	Warianty danych wejściowych	31
5.5	Konfiguracja treningu modeli	31
5.6	Metryki oceny	32
5.7	Procedura eksperymentalna	32
6	Wyniki badań i ich analiza	33
6.1	Wyniki uczenia modeli YOLO	33
6.2	Porównanie wariantów modeli	33
6.3	Wpływ reprezentacji obrazu na skuteczność detekcji	33
6.4	Analiza błędów klasyfikacji i detekcji	33
6.5	Ocena działania systemu w scenariuszu użytkowym	33
6.6	Dyskusja ograniczeń wyników	34
7	Podsumowanie i wnioski końcowe	35
7.1	Podsumowanie wykonanych prac	35
7.2	Ocena realizacji celu pracy	35
7.3	Najważniejsze wnioski	35
7.4	Możliwe kierunki rozwoju	35

Dokumentacja techniczna	45
Spis skrótów i symboli	47
Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)	49
Spis rysunków	51
Spis tabel	53

DRAFT

DRAFT

Rozdział 1

Wstęp

To powinien być rozdział „ustawiający” całą pracę. Powinny się tu znaleźć:

- kontekst: technologie wspomagające osoby z niepełnosprawnością wzroku,
- problem: rozpoznawanie dotykanych obiektów 3D w czasie zbliżonym do rzeczywistego,
- cel pracy,
- zakres pracy,
- wkład własny,
- krótka charakterystyka rozdziałów.

1.1 Motywacja podjęcia tematu

Tu piszę, dlaczego temat ma sens: samodzielność osób niewidomych, rozpoznawanie obiektów, ograniczenia obecnych rozwiązań.

1.2 Cel i zakres pracy

Cel np.: „Celem pracy jest opracowanie i eksperymentalna ocena systemu wspomagającego rozpoznawanie dotykanych obiektów 3D z wykorzystaniem metod detekcji obiektów na obrazie.”

1.3 Główne założenia i ograniczenia

Tu można wpisać: iPhone jako urządzenie wejściowe, obiekty drukowane 3D, sześć klas, detekcja w obrazie, komunikacja z serwerem, informacja zwrotna audio.

1.4 Wkład autora

Bardzo ważne, bo wymagania tego chcą. Wkład: przygotowanie datasetu, implementacja klienta/serwera, trening modeli, eksperymenty, analiza wyników.

1.5 Struktura pracy

Krótki opis, co jest w każdym rozdziale.

DRAFT

Rozdział 2

Analiza problemu i przegląd literatury

Zagadnienie rozpoznawania obiektów przez osoby niewidome wymaga uwzględnienia nie tylko aspektów technicznych, ale też społecznych i psychologicznych – tego, jak użytkownik poznaje otoczenie i jakie informacje z niego wyciąga. Osoby widzące zazwyczaj nie mają problemów z rozpoznawaniem kształtów, kolorów, orientacji czy położenia obiektów w przestrzeni, natomiast osoby niewidome i słabowidzące w większym stopniu wykorzystują inne zmysły i technologie wspomagające, aby móc zbudować mentalną reprezentację otoczenia.

Ze względu na to, projektowanie i wykonanie systemu rozpoznawania obiektów nie może skupiać się wyłącznie na aspektach technicznych, ale należy również uwzględnić perspektywę użytkownika docelowego. Należy zrozumieć, w jaki sposób informacja przekazywana przez komputer może wesprzeć proces poznawania otoczenia. W niniejszym rozdziale przedstawiono perspektywę osób niewidomych w kontekście rozpoznawania obiektów przestrzennych, a następnie omówiono wybrane technologie wspomagające osoby z niepełnosprawnością wzroku i metody komputerowej detekcji obiektów z wykorzystaniem zróżnicowanych danych wejściowych, takich jak obrazy RGB, mapy głębi czy dane LiDAR. Szczególną uwagę poświęcono rodzinie modeli YOLO, wykorzystywanej w niniejszej pracy. W kolejnej części rozdziału przedstawiono metryki wykorzystywane do oceny jakości detekcji obiektów, a następnie podsumowano przeprowadzoną analizę.

2.1 Perspektywa osób niewidomych w rozpoznawaniu obiektów przestrzennych

Przy projektowaniu systemu przeznaczonego dla osób niewidomych należy wziąć pod uwagę różnice w percepcji przestrzennej pomiędzy osobami widzącymi a niewidomymi. W

literaturze zagadnienie to jest analizowane zarówno z perspektywy technicznej, psychologicznej, jak i pedagogicznej. Badania nad percepcją osób niewidomych pozwalają lepiej zrozumieć ich sposób poznawania otoczenia. Costa et al. w swojej pracy, na podstawie studium przypadku, pokazują, że rozpoznawanie obiektu jest całym procesem stopniowego pozyskiwania i integrowania informacji. Zaznaczają, że w przypadku osób widzących spojrzenie zazwyczaj pozwala zrozumieć obiekt całościowo, natomiast w przypadku osób niewidomych poznanie odbywa się sekwencyjnie, etapami. Duże znaczenie mają więc cechy obiektu, które są wyraźnie dostępne w eksploracji dotykowej: prostota, regularność, krańdździe oraz faktura. Wspierają one budowę mentalnej reprezentacji przestrzennej obiektu. Osoba niewidoma może opierać ją nie tylko na aktualnej informacji dotykową, ale również wcześniejszych doświadczeniach, kontekście i wiedzy o świecie. Autorzy ukazują, że brak wzroku nie oznacza prostego zastąpienia go przez inne zmysły – dotyk, słuch, pamięć i doświadczenia dostarczają informacji innego rodzaju oraz są wykorzystywane w odmienny sposób [4].

Rozwinięcie tej myśli znajdujemy w pracy autorstwa Figueiras i Arcavi, którzy pokazują jak osoba niewidoma uczy się matematyki i w jaki sposób komunikuje się z nauczycielem. Autorzy opisują przebieg lekcji prowadzonych przez niewidomego nauczyciela dla trzech uczniów, z czego dwóch było w pełni niewidomych, a jeden miał częściową utratę wzroku. Szczególnie istotne okazują się zapisy rozmów pomiędzy nauczycielem a uczniami, które ukazują wcześniej wspomniane etapowe poznawanie obiektu. Podczas omawiania stożka nauczyciel bazuje na wcześniejszych doświadczeniach i wiedzy uczniów, a także na ich wyobrażeniach przestrzennych.

W kolejnej części artykułu autorzy ukazują w jaki sposób nauczyciel prowadzi uczniów przez analizę wykresu funkcji kwadratowej. Nauczyciel celowo odracza rozwiązanie algebraiczne i prowadzi uczniów przez proces rozumowania. Początkowo uczestnicy rozważają położenie jednego z pierwiastków, następnie wierzchołka i osi symetrii. Dopiero po tym, gdy uczniowie przeprowadzili rozumowanie, nauczyciel pozwala na rozwiązanie algebraiczne. Pokazuje to, że w przypadku, gdy nie mamy dostępu do globalnej reprezentacji obrazu, uwypukla się znaczenie jawnych opisów relacji pomiędzy charakterystycznymi cechami obiektu, odwołania do wcześniejszych reprezentacji i uporządkowanie procesu rozumowania [12]. W kontekście projektowanego systemu sugeruje to, że sama informacja o klasie rozpoznanego obiektu nie jest wystarczająca, a zasadne może być również przekazywanie informacji o jego cechach charakterystycznych i zachodzących między nimi relacjach.

Odchodząc od ściśle teoretycznych i obserwacyjnych badań, w literaturze znajdujemy również przykłady artykułów naukowych, które bezpośrednio wskazują problemy osób niewidomych. Güler i Ürey przeprowadzili badanie jakościowe obejmujące 13 uczniów szkoły dla osób z niepełnosprawnością wzroku oraz ich 10 nauczycieli. Dotyczyło ono wyzwań i potrzeb uczniów przy nauce przedmiotów przyrodniczych. Istotnym wnioskiem

z dokumentu jest wskazanie, że nauczyciele częściej korzystali z podręczników pisanych alfabetem Braille'a oraz tabliczek i ryłców do zapisu brajlowskiego, aniżeli z materiałów przestrzennych, modeli 3D czy eksperymentów. Jeden z uczniów wskazywał, że ma problem z wyobrażaniem sobie reprezentacji przestrzennych słów, twierdząc, że gdy słyszy np. słowo "piramida" nie potrafi zbudować w umyśle odpowiadającego mu obrazu. Nauczyciele wskazują również, że tureccy uczniowie dotknięci niepełnosprawnością wzroku nie mają przewidzianego osobnego toku nauczania. Są więc zmuszeni do konkurowania z widzącymi rówieśnikami tak, jakby byli w stanie wyuczyć się wszystkiego w ten sam sposób. Zaznaczają też, że zagadnienia związane z naukami przyrodniczymi są często abstrakcyjne, co utrudnia ich zrozumienie, a sam proces nauki powinien być dostosowany do potrzeb uczniów z niepełnosprawnością wzroku [13]. Podobne problemy związane z brakiem materiałów edukacyjnych, koniecznością dostosowania istniejących, wydłużonym czasem przygotowania zajęć i dostosowania komunikacji odnotowano również w badaniach dotyczących nauki języka angielskiego [2].

Kolejnym zagadnieniem dotyczącym perspektywy osób niewidomych, na które zwrócili uwagę Li et al., jest sposób, w jaki różne źródła informacji: dotyk, opis dźwiękowy, wcześniejsze doświadczenia; wspierają rozumienie obiektów. Zaznaczają, że osoby niewidome nie bazują jedynie na pojedynczym źródle informacji, ale łączą informacje pochodzące z różnych modalności. Badana grupa osób wskazała, że poprzez dotyk mogą zrozumieć kształt obiektu, jego położenie, formę i szczegóły przestrzenne, ale by w pełni zrozumieć kontekst, historię lub znaczenie obiektu, potrzebują dodatkowego opisu audio. Dodatkowo, autorzy zwracają uwagę na to, że osoby niewidzące różnią się między sobą pod względem preferencji i sposobów poznawania obiektów. Zaznaczają, że różnią się one w zależności od stopnia niepełnosprawności wzroku oraz tego, czy osoba jest niewidoma od urodzenia, czy utraciła wzrok w późniejszym okresie życia [17]. Na tej podstawie można przyjąć, że projektowany system nie powinien ograniczać się do jednego sposobu komunikacji, ale pozwolić na personalizację i dostosowanie do preferencji konkretnego użytkownika

Przytoczone badania wskazują, że rozpoznawanie obiektów przez osoby niewidome nie jest pojedynczym aktem identyfikacji, ale etapowym procesem poznawczym. Eksploracja dotykowa pozwala na poznanie lokalnych części obiektu: krawędzi, powierzchni, faktury i orientacji, ale to wcześniejsze doświadczenia i przekaz słowny pozwalają na zbudowanie jego pełnej reprezentacji mentalnej. Jednocześnie, mała ilość dedykowanych materiałów edukacyjnych mocno utrudnia procesy nauczania – zwłaszcza przy pojęciach abstrakcyjnych.

Wynika z tego, że system wspomagający nie powinien ograniczać się jedynie do przekazywania nazw obiektów, ale wspierać użytkownika w całym procesie jego rozpoznawania. Informacja zwrotna powinna być jednoznaczna, możliwa do dostosowania do potrzeb użytkownika i wspierać go w naturalnym procesie eksploracji dotykowej. Te wnioski są

podstawą do analizy istniejących rozwiązań technologicznych przedstawionej w kolejnym podrozdziale.

2.2 Technologie wspomagające osoby z niepełnosprawnością wzroku

Analiza perspektywy osób niewidomych pozwala zauważyć nie tylko to, że wymagają oni jedynie odmiennych sposobów poznawania przestrzeni wokół nich, ale również wsparcia w wielu przypadkach codziennego życia. Takim wsparciem może zostać prosta laska, ale w bardziej skomplikowanym środowisku musi to być pies przewodnik lub drugi człowiek. Można jednak zastąpić ich technologią, która ciągle rozwijana daje coraz lepsze możliwości.

2.2.1 Rozwój technologii asystujących

Naukowcy od dawna widzieli w rozwoju techniki okazję do wsparcia osób z jakąkolwiek niepełnosprawnością. Jednym z pierwszych urządzeń wspomagających niesłyszących był aparat słuchowy opracowywany przez Millera Reese'a Hutchisona od około 1895 roku [25]. Rozwiązaniem, które zapowiadało późniejszy rozwój elektronicznych systemów wspomagających był system POSSUM (ang. *Patient Operated Selector Mechanism*) opracowany początkiem lat 60. XX wieku. Umożliwiał on osobom z poważną niepełnosprawnością ruchu sterowanie maszyną do pisania [23]. Nie można jednak mówić o cyfrowych systemach wspomagających przed pojawieniem się elektronicznych komputerów. W latach 60. i 70. XX wieku zaczęły pojawiać się bardziej zaawansowane systemy wspomagające oparte o cyfrowe jednostki obliczeniowe. Przykładem jednego z nich może zostać Tufts Interactive Communicator, który umożliwiał niemym z ograniczeniami ruchowymi komunikację z otoczeniem poprzez wybieranie znaków za pomocą mechanizmu skanowania, a następnie prezentację pełnego komunikatu na wyświetlaczu w formie wydruku [42]. Późniejszy przykład podobnego systemu był wykorzystywany przez wybitnego fizyka Stephena Hawkinga który, pomimo swojej postępującej niepełnosprawności ruchowej, mógł sprawnie komunikować się z otoczeniem poprzez niewielkie ruchy policzka [15].

Równocześnie rozwijały się technologie wspomagające osoby z niepełnosprawnością wzroku, w którym dane przekształcano na formę możliwą do odbioru przez inne zmysły. Kamieniem milowym okazało się urządzenie Optacon (ang. *Optical to Tactile Converter*) opatentowane w 1966 roku przez Johna G. Linvilla. Umożliwiała ono przekształcenie obrazu na vibracje matrycy, które mogły być odczytane przez użytkownika za pomocą dotyku [20]. W kolejnych dekadach rozwijano m. in. systemy rozpoznawania znaków, syntezy mowy, czytniki ekranów i urządzenia mobilne, stopniowo zwiększając i umożliwiając osobom niewidomym dostęp do informacji cyfrowych.

Współczesny rozwój takich rozwiązań dokonuje się dzięki stopniowej miniaturyzacji elektroniki i popularyzacji urządzeń osobistych. Według prognozy MarketsandMarkets rynek urządzeń ubieralnych ma wzrosnąć do 238,71 mld dolarów w 2032 [24]. Dane te są przesłanką do wniosku, że w najbliższych latach możemy spodziewać się dynamicznego rozwoju małych urządzeń wyposażonych w czujniki, układy umożliwiające komunikację i mikroprocesory zdolne do szybkiego, lokalnego przetwarzania danych. Te same podzespoły mogą umożliwiać również rozwój systemów wspomagających osoby niewidome, a wykorzystanie szybkich sieci bezprzewodowych i nagły rozwój uczenia maszynowego pozwoli na szybsze tworzenie i udoskonalanie działających w czasie rzeczywistym algorytmów.

2.2.2 Standardy i regulacje prawne dotyczące dostępności

Rozwój technologiczny nie był jedynym czynnikiem, który pchnął do przodu upowszechnienie się systemów wspomagających. Istotną rolę odegrały również regulacje prawne i standardy, które nakładały obowiązek zapewnienia dostępności systemów informatycznych dla osób z niepełnosprawnością. Bardzo istotnym krokiem było opublikowanie w 1999 roku przez World Wide Web Consortium (W3C) pierwszej wersji wytycznych WCAG (ang. *Web Content Accessibility Guidelines*) określających, w jaki sposób powinny być projektowane treści internetowe, by były one dostępne dla osób z różnymi rodzajami niepełnosprawności [45]. Kolejne wersje wytycznych uporządkowały zasady wokół czterech głównych: postrzegalności, funkcjonalności, zrozumiałości i solidności. Najnowsza wersja standardu WCAG 2.2 została opublikowana 5 października 2023 [46]. Obecnie trwają prace nad kolejną wersją WCAG 3.0.

Znaczenie dostępności zostało następnie zwiększone dzięki wdrożeniu kroków prawnych. Artykuł 9 Konwencji ONZ o prawach osób z niepełnosprawnościami nakłada na państwa członkowskie obowiązek zapewnienia dostępu do informacji, komunikacji, infrastruktury i technologii do potrzeb osób z niepełnosprawnością [41]. Unia Europejska wprowadziła w 2016 roku dyrektywę 2016/2102, która określa wymagania dotyczące dostępności stron internetowych i aplikacji mobilnych podmiotów sektora publicznego [9]. W Polsce wprowadzono te rozporządzenie ustawą z dnia 4 kwietnia 2019 roku o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów publicznych [33]. Od 28 czerwca 2025 roku w Polsce obowiązuje również Europejski Akt o Dostępności, który nakłada wymagania na wybrane usługi i produkty, takie jak urządzenia mobilne, komputery, czytniki książek, handel elektroniczny, bankowość i transport [10, 32]. Szczegółowe wymagania techniczne dla produktów i usług ICT określa norma EN 301 549, która jest stosowana w całej Unii Europejskiej [8].

Dla osób niewidomych te regulacje mają szczególne znaczenie w kontekście współpracy czytników ekranowych z interfejsami, możliwości użycia bez użycia wzroku i jednoznacznego przekazywania informacji. Rozwój prawa sprawił więc, że dostępność cyfrowa nie

pozostała w fazie teoretycznej, ale stała się wymogiem, a przez to istotnym czynnikiem technologii wspomagających.

2.2.3 Współczesne systemy wspomagające osoby niewidome

Wraz z rozwojem technologii informatycznej i postępującym zwiększaniem dostępności cyfrowej zaczęły powstawać kompletne systemy wspomagające osoby niewidome, których rozmiary i możliwości zazwyczaj nie krępowały codziennego funkcjonowania. Dziś, wiele z nich wykorzystuje uczenie maszynowe i kamery do analizy przestrzeni wokół użytkownika. Przykład ich użycia zaproponowali Potdar, Pai i Akolkar, którzy oparli swoją pracę na konwolucyjnych sieciach neuronowych (CNN) i kamerze. Zaznaczyli, że jednym z ich celów było, aby osoba niewidoma nie musiała wchodzić w bezpośredni kontakt z obiektem przed sobą, aby zminimalizować ryzyko uszczerbku na zdrowiu lub utraty życia. Sieć zastosowana przez autorów zawierała warstwy odpowiadające za wyodrębnianie cech, lokalizację i klasyfikację obiektów. System umożliwia rozpoznawanie 200 klas obiektów obejmujących ludzi, pojazdy, zwierzęta i elementy wyposażenia pomieszczeń. Sens działania systemu jest prosty: użytkownik naciska przycisk, kamera rejestruje obraz, a następnie model CNN rozpoznaje obiekty i przekazuje użytkownikowi informacje o nich. Niestety, założenie o naciśnięciu przycisku wyklucza możliwość ciągłego rozpoznawania obiektów, co może prowadzić np. do wypadków z szybko poruszającymi się pojazdami. Autorzy zaznaczają, że ich wyniki potwierdzają użyteczność systemu, ale jednocześnie mówią o utracie jakości przy małych obiektach. Sugerują, że zwiększenie ilości warstw konwolucyjnych może poprawić skuteczność, ale jednocześnie spowolni działanie systemu. Te wnioski pokazują, że należy zachować kompromis pomiędzy skutecznością, a prędkością działania systemu [27].

Rozszerzeniem idei wykorzystania sieci neuronowych do wsparcia osób niewidomych jest system MagicEye opracowany przez zespół pod przewodnictwem Sethuramana. Tym razem twórcy oparli się na idei stworzenia urządzenia ubieralnego w formie inteligentnych okularów z kamerą. Podobnie jak w przypadku poprzedniego systemu, po wejściu w interakcję z urządzeniem obraz z kamery jest analizowany przez model sieci neuronowej, a następnie wyniki detekcji są przekazywane użytkownikowi w formie dźwiękowej. Model oparto o architekturę YOLOv5, wybranej ze względu na szybkość działania, dokładność i niewielkiego rozmiaru sieci. Trening oparty był o wybrany podzbiór Google Open Images liczący ponad 13 200 obrazów, obejmujący 35 klas obiektów. Dodatkowo, w systemie zaimplementowano mechanizmy rozpoznawania twarzy, nominałów banknotów, odbiornik GPS i czujnik zbliżeniowy [35].

Kolejnym krokiem w rozwoju technologii asystujących jest już nie tylko rozpoznanie obiektów, ale przekazanie użytkownikowi informacji dotyczących kontekstu całej sceny. Takie rozwiązanie znajdujemy w pracy Baiga et al., którzy zaproponowali system oparty o Raspberry Pi 4 i model sztucznej inteligencji. Dzięki temu, nie byli ograniczeni do

skończonego zbioru klas obiektów, ale dzięki użyciu dużego modelu vision-language mogli generować opisy całej sceny i przekazywać je użytkownikowi w formie audio. Dodatkowo, w systemie zaimplementowano rozpoznawanie twarzy i detekcję odległości od przeszkód. Zgodnie z założeniami, system działał w czasie rzeczywistym, ale radził sobie znacząco gorzej w złych warunkach oświetleniowych [1]. Ta praca rozwiązuje problem poruszony w podrozdziale 2.1, mówiący o tym, że sama informacja o obiekcie może być niewystarczająca. Tutaj twórcy skupili się na przekazaniu jak najpełniejszej informacji o przestrzeni w której znajduje się użytkownik. Warto jednak zauważyć, że skorzystanie z dużego modelu wymaga ciągłego połączenia z Internetem, a lokalne działanie systemu jest ograniczone do rozpoznawania twarzy i detekcji pobliskich przeszkód.

Poprzednio omawiane systemy skupiały się na rozpoznawaniu obiektów w określonej scenie, natomiast głównym celem systemu zaproponowanego przez Wang et al. jest pomoc użytkownikowi w omijaniu przeszkód w przestrzeni. Ponieważ rozwiązanie ma działać jako ciągła pomoc nawigacyjna, nie powinno być aktywowane przyciskami, a zamiast tego stale analizować obraz i – w razie potrzeby – informować użytkownika o potencjalnym zagrożeniu. Autorzy przedstawili metodę detekcji YOLO-OD do której wykorzystali model o architekturze inspirowanej siecią YOLOv8 rozszerzoną o dodatkowe bloki odpowiedzialne za zwiększenie skuteczności detekcji w różnych warunkach. Sam model na publicznie dostępnym zbiorze danych osiągnął wartość średniej precyzji na poziomie 30.02%, co zdaniem autorów wskazuje, że zaproponowane rozwiązanie może być wartościowe w kontekście wspomagania osób niewidomych [44].

Odrębnym zagadnieniem, na jakie należy zwrócić uwagę przy tworzeniu systemów wspomagających, jest sposób przekazywania informacji. Wcześniej omawiane rozwiązania bazowały na prostym przekazie dźwiękowym, opartym o buzzer lub syntezytor mowy. Dzierzgowska et al. zaproponowali metodę audio-graficzną, łączącą interaktywną prezentację wykresów funkcji matematycznych z opisami dźwiękowymi. Użytkownik, aby uzyskać kolejne informacje o wybranej części wykresu, musiał wykonać pojedyncze, podwójne lub potrójne stuknięcie na elemencie. Badanie przeprowadzone na grupie 54 osób wskazało, że zaproponowana metoda umożliwiła zmniejszenie obciążenia, frustracji, a w przypadku części zadań pozwoliło na skrócenie czasu ich wykonania. Wyniki wskazują, że skuteczność systemu wspomagającego nie zależy jedynie od stopnia możliwości rozpoznawania obiektów, ale również od poprawnego i odpowiedniego zaprojektowania przekazu informacji zwrotnej [7].

Współczesne technologie wspomagające uzupełniają tradycyjne metody wsparcia osób niewidomych, oferując coraz bardziej złożone systemy i algorytmy. Wspólnym mianownikiem większości z nich jest wykorzystanie małych kamer do pozyskiwania obrazu, który następnie jest analizowany przez modele uczenia maszynowego, a wyniki są prezentowane użytkownikowi. Większość opisywanych systemów koncentruje się jednak na wsparciu w codziennym funkcjonowaniu – poruszaniu się w przestrzeni, wykrywaniu przeszkód czy

rozpoznawaniu twarzy i banknotów. Mało systemów skupia się jedynie na obiektach aktualnie eksplorowanych dotykowo przez użytkownika. W tym kontekście stworzenie systemu dedykowanego nauce przestrzennej wydaje się być istotnym uzupełnieniem istniejących rozwiązań.

2.3 Problem rozpoznawania obiektów 3D w interakcji dotykowej

Problemem, jaki należy koniecznie poruszyć, jest rozpoznanie obiektów 3D w interakcji dotykowej, ponieważ zachodzi w tym miejscu istotna różnica od systemów opisywanych we wcześniejszym podrozdziale (2.2.3). W odróżnieniu od nich, głównym celem projektowanego systemu nie jest rozpoznanie obiektów przed użytkownikiem, a tych z którymi wchodzi w interakcję. Mamy więc do czynienia z sytuacją, w której kamera nie jest zwrócona na to co znajduje się przed użytkownikiem, ale na niego samego, a w szczególności na jego dłoń. W tym kontekście klasyczne podejście do detekcji może być niewystarczające ze względu na to, że spora część obiektu może być przez nie zasłonięta, a sam kształt może być trudny do rozpoznania nawet dla osoby widzącej. W tym miejscu warto przytoczyć kilka prac, które poruszają podobne zagadnienia.

Pierwszą z nich jest badanie przeprowadzone przez Shan et al., którzy sprawdzali zdolność modelu do rozpoznawania dłoni w kontakcie z obiektami. W tym celu przygotowali zbiór danych ponad 100 tys. obrazów, na których oznaczono ponad 180 tys. dłoni i ponad 110 tys. obiektów. Celem zaproponowanego modelu nie było jednak określenie klasy dotykane go przedmiotu, a jak najlepsze wskazanie pozycji dłoni i jej orientacji. Dzięki temu, możliwe było wyznaczenie nie tyle położenia przedmiotu, a formy relacji, w jaką wchodził z dłonią. Detekcja została oparta o model Faster R-CNN, który dla pełnej predykcji wymagającej poprawnego wkrycia dłoni i obiektu uzyskał 38,5 AP50 [36]. Pokazuje to, że w przypadku wykrywania trzymanych obiektów należy zwrócić uwagę na ich otoczenie, gdyż może dojść do sytuacji, gdy model poprawnie rozpozna obiekt, ale nie będzie on trzymany przez użytkownika, co może prowadzić do błędnych wyników.

Kolejnym aspektem na jaki należy zwrócić uwagę jest możliwość zasłonięcia części obiektu przez dłoń użytkownika. W odróżnieniu od poprzedniej pracy, Zhang et al. skupili się na rekonstrukcji trójwymiarowej przedmiotów trzymanych w dłoniach. Podczas uczenia wykorzystywali dane z wielu widoków, aby ograniczyć wpływ zasłaniania obiektu przez dłoń. Autorzy wskazali dwa źródła niepełności informacji: zasłonięcie części przedmiotu przez dłoń lub niewidoczność części obiektu wynikająca z obserwowania go tylko pod jednym kątem. Wskazuje to, że w pojedynczej klatce nawet dobrze widoczny przedmiot może nie ujawniać wszystkich swoich cech [48]. Przez to istotne jest, aby zbiór danych treningowych zawierał bardzo zróżnicowane kombinacje ułożenia dłoni i obiektu. Warto zwrócić

uwagę na to, że możliwa jest również analiza obiektu z kilku ujęć, co może pozwolić na odtworzenie jego widocznej części w przestrzeni trójwymiarowej. W literaturze znajdujemy również propozycje analizy sekwencji obrazów, która pozwoli na ograniczenie problemu zasłaniania obiektu, a możliwe, że również wykluczy niektóre przypadki błędnej predykcji. Jedną z prac poruszających ten problem jest badanie zespołu pod przewodnictwem Tekina. Zaproponowali oni system analizujący sekwencje obrazów RGB rejestrowanych przez użytkownika z perspektywy pierwszej osoby. Model jednocześnie określa położenie dłoni i obiektu, rozpoznaje jego klasę i wykonywaną czynność. Kolejne klatki są następnie analizowane przez moduł rekurencyjny, badający zmiany położenia wykrywanych dłoni i obiektów. Autorzy zaznaczają, że takie podejście pozwoliło im na poprawę skuteczności rozpoznawania czynności z 85,56% do 96,99% w porównaniu z analizą pojedynczych klatek. System działał również w czasie rzeczywistym z prędkością około 25 klatek na sekundę [37]. Widzimy więc, że przy rozpoznawaniu obiektów istotne jest nie tyle samo ich wykrycie, a przetworzenie ich zbioru w czasie, aby uzyskać jak najpełniejszą informację.

Na problem rozpoznawania obiektów można również spojrzeć z perspektywy robotyki. Możliwa jest eksploracja obiektu bez użycia kamer, a jedynie poprzez dotyk. Mimo że w tym przypadku nie mamy bezpośredniego odniesienia do osób niewidomych, to warto zwrócić uwagę na inne sposoby akwizycji danych. Ramię robota stworzonego przez zespół Xu et al. wyposażono w czujnik dotyku, a następnie umożliwiono mu eksplorację 10 przedmiotów ze zbioru YCB. Każde dotknięcie tworzyło wpis w lokalnej chmurze punktów, która następnie była wykorzystywana do predykcji lub określenia kolejnego ruchu pozwalającego na lepsze rozróżnienie przedmiotów. Autorzy wskazali, że już kilka charakterystycznych punktów pozwalało na odróżnienie obiektów [47]. Sugeruje to, że kamery nie są jedynym sposobem na pozyskanie informacji, a ciekawym kierunkiem badań może być analiza chmury punktów powstałej w wyniku dotyku lub skanowania przestrzeni. Odpowiednikiem wspomnianego robota może być rękawica sensoryczna, która będzie zbierała dane nie tylko o samym obiekcie, ale również o rękach użytkownika. Zhang et al. opracowali właśnie takie rozwiązanie, które na podstawie zbieranych danych mogło rozpoznać 16 klas obiektów z dokładnością ponad 95% [49]. Ukazuje to, że do rozpoznania obiektów nie musimy wykorzystywać wyłącznie obrazów, a możliwe jest połączenie różnych źródeł danych, takich jak obraz, głębia czy informacje dotykowe.

2.4 Metody detekcji obiektów w obrazach

Mimo że dzisiejsza technika detekcji obiektów w obrazach opiera się głównie na sieciach neuronowych, istnieją różne podejścia do tego problemu. Należy wspomnieć o klasycznych metodach wizji komputerowej, wykorzystujących cechy samego obrazu, takie jak krawędzie, kolory czy histogramy [51]. Ten rozdział przedstawia przegląd najważniejszych metod detekcji obiektów.

2.4.1 Sformułowanie problemu detekcji obiektów

Detekcja jest zadaniem trudniejszym od klasyfikacji obrazu, ponieważ oprócz określenia, jakie obiekty znajdują się na obrazie, musi również wskazać ich położenie. Sama klasyfikacja opiera się na przypisaniu jednej lub kilku etykiet do całego obrazu, co w wielu przypadkach może być niewystarczające. Lokalizacja rozszerza klasyfikację o wyznaczenie ramki ograniczającej (ang. *bounding box*) położenie obiektu, najczęściej zakładając pojedynczą instancję. Zadaniem detekcji jest więc odnalezienie wszystkich istotnych obiektów na obrazie, a następnie przypisanie odpowiadających im klas i określenie ich położenia [29]. Wynik działania detektora może być przedstawiony jako zbiór predykcji:

$$D = \{(c_i, b_i, p_i)\}_{i=1}^N, \quad (2.1)$$

gdzie N jest liczbą wykrytych obiektów, c_i to przewidywana klasa i -tego obiektu, b_i opisuje jego ramkę, a p_i jest wartością pewności predykcji.

Ramka ograniczająca najczęściej jest prostokątem opisanym współrzędnymi dwóch przeciwległych narożników lub częściej współrzędnymi środka, szerokości i wysokości:

$$b_i = (x_i, y_i, w_i, h_i), \quad (2.2)$$

gdzie x_i i y_i określają położenie środka ramki, a w_i i h_i jej szerokość oraz wysokość. Zależnie od implementacji każda z tych wartości może być wyrażona w pikselach lub znormalizowana względem rozmiaru obrazu. Drugie podejście uniezależnia reprezentację ramki od rozdzielczości obrazu wejściowego i jest powszechnie stosowane przy adnotacjach i predykcji.

Wartość pewności predykcji, w zależności od implementacji, określa stopień przekonania modelu, że dany obszar zawiera obiekt przewidywanej klasy. Jeżeli ta wartość nie przekracza przyjętego progu, wynik wykonanej predykcji może zostać odrzucony. Jest to więc istotny parametr, jednak nie przesądza on poprawności predykcji. Przy ocenie należy również spojrzeć na zgodność klasy przewidywanej ze stanem faktycznym oraz na stopień pokrywania się ich ramek ograniczających (ang. *bounding box*). Zagadnienia te są szczegółowo opisane w podrozdziale 2.7, dotyczącym metryk oceny detekcji.

Zadaniem powiązanym z detekcją obiektów jest segmentacja instancji, której celem oprócz wyznaczenia ramki i klasy każdego obiektu jest określenie ich masek pikselowych. Umożliwia to dokładne określenie kształtu obiektów, ale wymaga przygotowania dokładniejszych danych uczących [14]. W przypadku projektowanego systemu zadaniem modelu jest wykrycie trzymanej bryły geometrycznej, określenie jej klasy, położenia oraz wartości pewności predykcji. Istotna przy tym jest nie tylko dokładność predykcji, ale szczególnie szybkość działania, gdyż wynik ma być przekazywany użytkownikowi w momencie jego

bezpośredniej interakcji z obiektem. W tym kontekście segmentacja instancji może nałożyć za duże wymagania obliczeniowe, a jej zastosowanie może nie wiązać się z istotną poprawą jakości.

2.4.2 Klasyczne metody wizji komputerowej

Zanim w detekcji obiektów upowszechniły się głębokie sieci neuronowe, wykorzystywano przede wszystkim ręcznie projektowane metody przetwarzania obrazów. Między innymi wykorzystywano progowanie, segmentację na podstawie koloru, detekcję krawędzi i analizę konturów. Pozwalało to na wyodrębnienie obszarów różniących się od tła i opisanie ich cech, takich jak rozmiar, proporcje czy kształt. W kontrolowanych warunkach te metody mogły być skuteczne, jednak obiekty musiały wyraźnie odróżniać się od tła.

Późniejsze, bardziej rozbudowane podejścia wykorzystywały deskryptory cech, jak histogramy zorientowanych gradientów (HOG) czy skalo-niezmiennicze przekształcenie cech (SIFT) [5, 22]. Deskryptory obliczano dla całego obrazu lub kolejnych jego fragmentów za pomocą przesuwanego okna, a następnie wyodrębnione cechy przekazywano do klasyfikatora, którym mogła być maszyna wektorów nośnych (SVM). Warto również zwrócić uwagę na kaskady klasyfikatorów, mające cechy zbliżone do cech Haara, pozwalające na szybkie wykrywanie obiektów, głównie wykorzystywane w metodzie Viola–Jonesa [43].

Największe zalety metod klasycznych ujawniają się w ich prostocie, stosunkowo niskim koszcie obliczeniowym oraz możliwości interpretacji kolejnych etapów działania. Niestety, wymagają one ręcznego wyboru cech i ciągłego dostosowywania parametrów, aby uzyskać jak najlepsze wyniki. Są również bardzo wrażliwe na zmieniające się warunki otoczenia – niewielkie zmiany w oświetleniu, tle czy częściowe zasłonięcie obiektu mogą skutkować krytycznym spadkiem skuteczności. Te ograniczenia uzasadniają wykorzystanie głębokich sieci neuronowych, które w wielu przypadkach radzą sobie lepiej i mogą same, automatycznie uczyć się cech z danych treningowych.

2.4.3 Detektory dwuetapowe

Z czasem rozwój metod uczenia głębokiego doprowadził do coraz częstszego zastępowania ręcznie projektowanych deskryptorów cech przez ich reprezentacje automatycznie określone przez CNN. Jedną z rodzin takich rozwiązań są detektory dwuetapowe. Ich działanie podzielone jest na dwa główne etapy: wyznaczenie obszarów zainteresowań, a następnie ich klasyfikacja i dopasowanie położenia ramek ograniczających.

Jednym z pierwszych detektorów dwuetapowych był model R-CNN wykorzystujący algorytm do wyznaczenia propozycji regionów, które następnie kolejno były przetwarzane przez sieć konwolucyjną. Niestety, prowadziło to do wielokrotnego sprawdzania tych samych fragmentów obrazu przez sieć, co skutkowało dużym kosztem obliczeniowym. Kolejna wersja, Fast R-CNN, wprowadziła jednokrotne przetwarzanie całego obrazu przez

sieć, a cechy były pobierane ze wspólnej mapy cech. Ograniczyło to liczbę wykonywanych obliczeń, ale wciąż generowano propozycje regionów za pomocą algorytmu. Dalszym rozwinięciem było wprowadzenie wersji Faster R-CNN, w którym zastąpiono algorytm modułem sieci propozycji regionów RPN (ang. *Region Proposal Network*), która analizuje mapę cech i wskazuje obszary zawierające potencjalne obiekty. Oba moduły korzystały ze wspólnych cech obrazu, co pozwoliło znacznie ograniczyć koszt obliczeniowy [31].

Detektory dwuetapowe mogą osiągać wysoką skuteczność i dokładność lokalizacji dzięki rozdzieleniu od siebie etapu generowania propozycji regionów od ich klasyfikacji i dopasowania ramek ograniczających. Niestety, ten podział wpływa na złożoność obliczeniową całego modelu, a w konsekwencji na jego czas działania. W wielu przypadkach może stanowić to istotną przeszkodę, szczególnie jeżeli wynik ma być otrzymywany w czasie rzeczywistym.

2.4.4 Detektory jednoetapowe

Ostatnia dekada przyniosła znaczny rozwój detektorów jednoetapowych, których rdzeń działania opiera się na przewidywaniu klas obiektów i ich ramek ograniczających w pojedynczym kroku. Za pierwszy nowoczesny przykład takiego podejścia uznaje się OverFeat z 2013 roku [34], jednakże dopiero modele wprowadzone po roku 2015 zyskały wysoką popularność i rozpoznawalność. Mimo że historycznie detektory jednoetapowe oferowały mniejszą dokładność od detektorów dwuetapowych, ich główną zaletą była szybkość działania, co umożliwiło ich wykorzystanie w systemach czasu rzeczywistego (ang. *Real-Time*, RT) [18, 51].

Jednym z pierwszych znaczących modeli wprowadzonym na rynek jest SSD (ang. *Single Shot MultiBox Detector*) zaprezentowany na konferencji ECCV w 2016 roku przez Wei Liu. Jego działanie opiera się na przewidywaniu klas obiektów i położenia ramek ograniczających na kilku mapach o różnej rozdzielczości. Większe mapy umożliwiają wykrywanie mniejszych obiektów, a głębsze o mniejszej rozdzielczości służą rozpoznawaniu obiektów większych. Model wykorzystuje domyślne ramki o różnych proporcjach i skalach, których współrzędne następnie koryguje. Taki mechanizm umożliwia wykrycie obiektów o różnych rozmiarach bez osobnego etapu generowania propozycji regionów [21]. Głównym ograniczeniem modelu SSD stała się zależność skuteczności detekcji od odpowiedniego doboru ramek, a także spadek dokładności przy wykrywaniu małych obiektów.

W podobnym czasie został zaprezentowany również model YOLO (ang. *You Only Look Once*) w którym zadanie detekcji przedstawiono jako pojedynczy problem regresji. Takie podejście doprowadziło do modelu, który przewiduje ramki ograniczające oraz prawdopodobieństwa klas w oparciu bezpośrednio o obraz wejściowy [29]. Uproszczenie i ujednolicenie procesu umożliwiło uzyskanie krótkiego czasu działania, co z kolei sprawiło, że modele YOLO znalazły zastosowanie w systemach czasu rzeczywistego. Kolejne lata

przyniosły rozwój rodziny modeli YOLO, w której znalazły się warianty o różnej liczbie parametrów i złożoności obliczeniowej, umożliwiając wybranie modelu dopasowanego do wymagań i ograniczeń systemu. Szczegółowy opis rodziny modeli YOLO znajduje się w podrozdziale 2.5.

2.4.5 Detektory oparte na architekturze transformera

Podjęciem alternatywnym do detektorów opartych na sieciach konwolucyjnych są modele oparte na architekturze transformera. Podstawowym elementem tego podejścia jest mechanizm samo-uwagi (ang. *self-attention*), który pozwala na analizę zależności pomiędzy różnymi fragmentami danych wejściowych. Podczas działania modelu Vision Transformer obraz wejściowy jest dzielony na mniejsze fragmenty, które następnie są reprezentowane jako sekwencja wektorów i przetwarzane przez warstwy transformera. Takie podejście pozwala na znajdowanie relacji pomiędzy odległymi od siebie elementami obrazu. Użycie transformerów wizyjnych wymaga również przygotowania dużych zbiorów danych treningowych tak, aby model mógł nauczyć się zależności globalnych [6].

Pierwszą propozycją zastosowania tego podejścia w detekcji obiektów jest model DETR (ang. *DEtection TRansformer*) zaprezentowany przez zespół będący częścią Facebook AI Research pod kierownictwem Cariona w 2020 roku. Jego podstawowa wersja opiera się na wyznaczeniu podstawowych cech przez CNN, a następnie przekazaniu ich do transformera zrealizowanego w architekturze enkoder-dekoder. W dekodzie wykorzystywany jest zbiór zapytań obiektowych (ang. *object queries*), na których podstawie model dokonuje równoległej predykcji klas obiektów i ich ramek ograniczających. Wynik detekcji jest traktowany przez model jako zbiór predykcji, które w trakcie uczenia są dopasowywane do obiektów referencyjnych poprzez dopasowanie dwudzielne (ang. *bipartite matching*) przy pomocy algorytmu węgierskiego. Pozwala to na uniknięcie użycia ramek domyślnych i algorytmu NMS (ang. *Non-Maximum Suppression*). W stosunku do klasycznych detektorów, DETR upraszcza potok detekcji, co pozwala na jego uczenie jako pojedynczego rozwiązania end-to-end. Niestety zalety płynące z użycia tego podejścia w detekcji wiążą się z wysokim kosztem treningu i trudniejszą optymalizacją modelu, szczególnie w przypadku wykrywania małych obiektów [3].

2.4.6 Porównanie metod detekcji

Przytoczone metody detekcji obiektów różnią się od siebie przede wszystkim sposobem wyznaczania położenia obiektów na obrazie, złożonością obliczeniową oraz ich stosunkiem pomiędzy dokładnością a szybkością działania. Zestawienie ich najważniejszych właściwości, ważnych z punktu widzenia systemu, przedstawiono w tabeli 2.1.

Zestawienie nie wskazuje jednego najlepszego rozwiązania, ale przedstawia zalety i wady poszczególnych metod. Wybór odpowiedniego podejścia zależy od wymagań oraz

Tabela 2.1: Porównanie wybranych metod detekcji obiektów

Metoda	Charakterystyka	Atuty	Ograniczenia
Metody klasyczne	Ręcznie projektowane cechy i osobne etapy przetwarzania	Niewielkie wymagania obliczeniowe, interpretowalność i możliwość działania w kontrolowanych warunkach	Konieczność ręcznego określania cech, wrażliwość na zmianę oświetlenia, tła, orientacji i częściowe zasłonięcie obiektu
Faster R-CNN	Dwuetapowe generowanie propozycji regionów, ich klasyfikacja i dopasowanie ramek	Wysoka skuteczność i dokładna określanie położenia obiektów	Większa złożoność obliczeniowa i opóźnienie wynikające z przetwarzania dwuetapowego
SSD	Jednoetapowe predykcje na mapach cech o różnych rozdzielczościach	Krótki czas odpowiedzi i możliwość wykrywania obiektów o różnych rozmiarach	Zależność od konfiguracji ramek domyślnych i gorsze wyniki dla małych obiektów
YOLO	Jednoetapowa predykcja klas i ramek ograniczających	Szybkie działanie, możliwość pracy w czasie rzeczywistym i dostępność wariantów modelu	Konieczny kompromis pomiędzy szybkością a skutecznością, szczególnie przy małych lub częściowo zasłoniętych obiektach
DETR	Bezpośrednia predykcja zbioru obiektów z wykorzystaniem transformera i mechanizmu uwagi	Modelowanie globalnych zależności w scenie i brak konieczności używania ramek domyślnych, ani algorytmu NMS	Długi trening i trudniejsza optymalizacja

ograniczeń sprzętowych i środowiskowych. Najważniejszym wymaganiem stawianym projektowanemu systemowi jest działanie w czasie rzeczywistym: rozpoznanie obiektu i przekazanie rzetelnych informacji użytkownikowi powinny następować w jak najkrótszym czasie, aby zagwarantować aktualność przekazywanych informacji. Dlatego rozwiązania klasyczne nie są dobrym wyborem przez ich niską odporność na zmiany środowiskowe, zaś detektory dwuetapowe – mimo że mają wysoką skuteczność – wymagają większej ilości kroków wiodących do uzyskania wyniku, a co za tym idzie, dłuższego czasu pomiędzy kolejnymi detekcjami. Dla systemu wspomagającego osoby niewidome prędkość działania jest jego cechą krytyczną. Czas reakcji powinien być możliwie krótki, przy zachowaniu odpowiedniej dokładności predykcji, a predykcje, które nie spełniają wymagań dokładności powinny być odrzucane. Wśród dostępnych modeli detekcji preferowany jest ten, który

zapewni najkrótszy czas odpowiedzi. W literaturze podkreślany jest kompromis mówiący, że należy dążyć do jak największej dokładności, ale nie kosztem czasu reakcji [28].

Spośród przedstawionych metod detekcji obiektów, najlepszym wyborem wydają się detektory jednoetapowe, a spośród nich szczególnie rodzina modeli YOLO. Pojedynczy krok detekcji pozwala na bezpośrednie przewidywanie klasy obiektu i jego położenia, co znacznie skraca czas potrzebny na uzyskanie wyniku. Cała rodzina modeli jest projektowana tak, aby zapewnić krótki czas inferencji, jednocześnie zapewniając duży wybór wariantów modeli. Pierwsza wersja YOLO osiągnęła prędkość predykcji około 45 FPS (ang. *frames per second*), a jej uproszczone warianty nawet 155 FPS, co pokazuje, że jest odpowiednim wyborem do zastosowania w systemie RT [29].

Wykorzystanie detektora opartego na architekturze transformera nie jest konieczne, ponieważ w tym przypadku nie jest wymagane jawne modelowanie globalnych zależności w scenie. Analizowana scena ma zawierać niewielką liczbę obiektów, a zadaniem modelu jest przede wszystkim ich rozpoznanie. Korzyści płynące z globalnego przetwarzania nie równoważą dodatkowego kosztu, który należy ponieść przy uczeniu i optymalizacji [3].

2.5 Rodzina modeli YOLO

Pierwsza wersja YOLO zadebiutowała wraz z publikacją artykułu *You Only Look Once: Unified, Real-Time Object Detection* przez Josepha Redmona, Santosha Divvalę, Rossa Girshicka i Ali Farhadi w 2016 roku i od tego czasu rodzina modeli YOLO należy do najbardziej rozpoznawalnych i popularnych detektorów jednoetapowych. Autorzy określili problem detekcji obrazu jako regresję, w której pojedyncza sieć neuronowa przewiduje zarówno ramki obiektów, jak i odpowiadające im prawdopodobieństwa klas [29]. Kolejne lata przyniosły rozwój kolejnych modeli rodziny YOLO przez różne zespoły badawcze, więc nie istnieje pojedyncza gałąź rozwoju rodziny. Jedną z głównych gałęzi jest rozwijana przez firmę Ultralytics, która w 2020 roku zaprezentowała YOLOv5, a w następnych latach jej kolejne generacje. W chwili opracowywania pracy najnowszą wersją jest YOLOv6 [16].

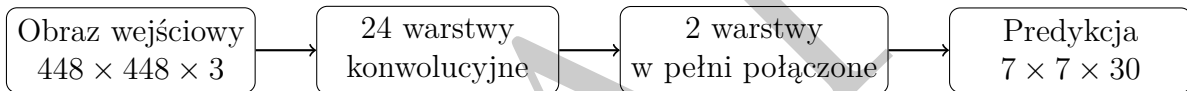
2.5.1 Architektura modeli YOLO

Mimo, że architektura każdej wersji modelu YOLO różni się od siebie, to idea pierwszej wersji pozostaje niezmienną. Założenie, że obraz jest analizowany w pojedynczym przebiegu sieci neuronowej, a wynik wyznaczany jest bez wcześniejszego wygenerowania propozycji regionów jest obecne w każdej kolejnej wersji. Sama architektura sieci mocno zmieniała się na przestrzeni kolejnych wersji, a porównanie kilku kolejnych modeli pozwala określić kierunek rozwoju całej rodziny.

Pierwszy model YOLO składał się z CNN złożonej z 24 warstw konwolucyjnych i dwóch warstw w pełni połączonych. Zadaniem warstw konwolucyjnych była ekstrakcja

cech z obrazu wejściowego, a warstwy końcowe generowały wynik detekcji. W modelu wykorzystano m. in. warstwy 1×1 do redukcji liczby kanałów oraz warstwy 3×3 [29].

Dla obrazów ze zbioru PASCAL VOC wejście sieci miało rozmiar 448×448 pikseli, które następnie były przekształcane w logiczną siatkę o wymiarach $S \times S$. W zastosowanej konfiguracji sieci przyjęto wartość $S = 7$ dzielącą obraz na 49 komórek. Wykrycie obiektu następowało w oparciu o komórkę, w której znajdował się środek jego ramki ograniczającej. Każda z komórek mogła przewidzieć $B = 2$ ramki, wartości ich pewności i prawdopodobieństwa $C = 20$ klas obiektów występujących w zbiorze. Rozmiar tensora był definiowany jako $S \times S \times (B \cdot 5 + C)$, co w omawianym przypadku dawało tensor o wymiarach $7 \times 7 \times 30$. Każda ramka opisana była za pomocą pięciu wartości: współrzędnych środka ramki, jej szerokości i wysokości oraz wartości pewności predykcji. Jednoczesne przewidywanie ramek i klas obiektów w pojedynczym kroku wyeliminowało wymaganie stosowania oddzielnych etapów generowania propozycji regionów i ich klasyfikacji. Schemat przedstawiony na rysunku 2.1 pokazuje uproszczoną architekturę pierwszego modelu YOLO, w której, pomimo stosunkowo prostej budowy, udało się połączyć w jeden model zarówno klasyfikację, jak i lokalizację obiektów.



Rysunek 2.1: Uproszczony schemat architektury pierwszego modelu YOLO [29].

Kolejne wersje YOLO przyniosły przede wszystkim zmiany w architekturze przez stosowanie coraz wydajniejszych modułów sieci, korzystanie z kilku skal obrazu i modyfikację sposobu generowania ramek ograniczających. Z czasem architekturę opartą na pojedynczej siatce zastąpiono architekturami wykorzystującymi wieloskalowe mapy cech. Zmodernizowano również sposób w jaki łączone są informacje z różnych warstw sieci, a także budowę głowic. Dzięki temu, współczesne modele można przedstawić za pomocą trzech głównych elementów: *backbone*, *neck* i *head*. Najważniejsze wersje i zmiany w ich architekturze zestawiono w tabeli 2.2.

Współczesna architektura modelu YOLO26 zachowała konwolucyjny rdzeń rodziny, ale znacząco różni się od pierwszej wersji modelu. Sieć podzielona jest na trzy główne części z których każda pełni inną funkcję. Pierwszą częścią jest *backbone*, który ma za zadanie ekstrakcję hierarchicznych reprezentacji cech z obrazu. Przechodzenie przez kolejne warstwy sieci wiąże się z zmniejszeniem rozdzielczości przestrzennej map cech, a jednocześnie zwiększeniem zdolności do rozpoznawania coraz bardziej złożonych i abstrakcyjnych cech. Kolejna część to *neck* odpowiedzialny za połączenie informacji pochodzących z różnych poziomów sieci, co jest kluczowe przy wykrywaniu obiektów o różnych wymiarach: cechy pochodzące z głębszych warstw są bardziej abstrakcyjne, a płytsze warstwy dostarczają informacji przestrzennych. To połączenie umożliwia modelowi prowadzenie detekcji na kilku skalach jednocześnie. Ostatnią częścią jest głowica detekcyjna (ang. *head*), która

Tabela 2.2: Wybrane etapy rozwoju architektury modeli YOLO. Opracowanie własne na podstawie [16, 29, 30, 40].

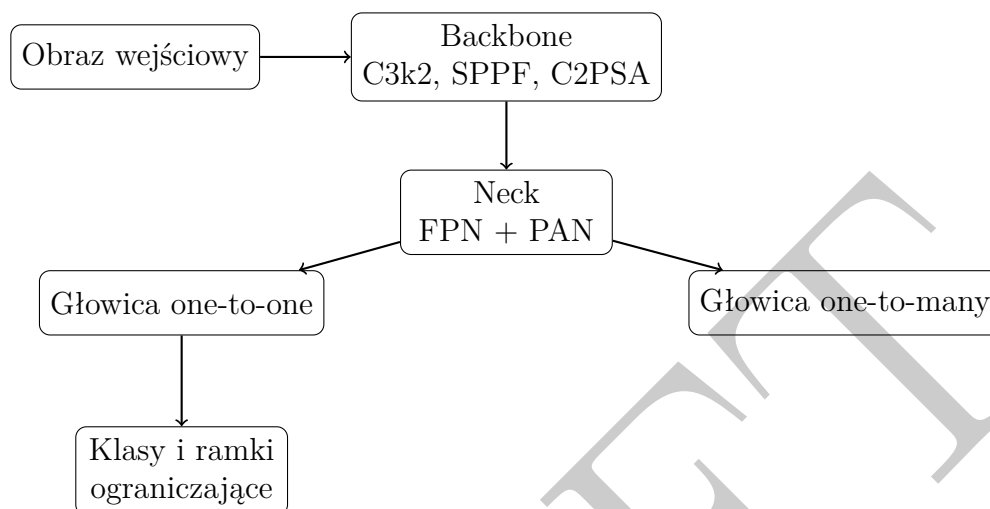
Generacja	Ekstrakcja cech	Sposób detekcji	Charakterystyczna zmiana
YOLO (v1)	Własna sieć CNN	Jedna siatka predykcji	Ujęcie detekcji jako pojedynczego problemu regresji
YOLOv3	Darknet-53	Detekcja na trzech skalach, ramki kotwiczące	Wprowadzenie wieloskalowej detekcji i architektury FPN
YOLOv5	CSP/C3	Wieloskalowa, ramki kotwiczące	Rozbudowanie fuzji cech o ścieżkę PAN i moduł SPPF
YOLOv8	C2f	Wieloskalowa, bez ramek kotwiczących	Przejsięcie na głowicę anchor-free i wykorzystanie DFL
YOLO11	C3k2, C2PSA	Wieloskalowa, bez ramek kotwiczących	Wprowadzenie kolejnych mechanizmów uwagi i zmian w blokach ekstrakcji cech
YOLO26	C3k2, C2PSA	Detekcja end-to-end	Domyślna detekcja bez NMS oraz rezygnacja z DFL

przewiduje klasy obiektów i położenie ich ramek ograniczających na podstawie dostarczonych z poprzedniej części map cech. Model YOLO26 wyposażony jest w dwie głowice decyzyjne: głowicę *one-to-many* i głowicę *one-to-one*. Pierwsza odpowiada za generowanie dużej liczby kandydatów obiektów, dostarczając gęstszego sygnału nadzorującego podczas uczenia, a druga uczy się przypisywać pojedynczą predykcję do każdego obiektu referencyjnego i jest domyślnie używana podczas inferencji [16].

Co ważne, podczas domyślnego wnioskowania model YOLO26 wykorzystuje głowicę *one-to-one*. Dzięki temu, że predykcja jest jednoznacznie przypisana model może dokonywać detekcji bez użycia algorytmu NMS, a co za tym idzie może realizować detekcję typu end-to-end. W sytuacjach, gdzie użycie algorytmu NMS jest dopuszczalne model może korzystać z głowicy *one-to-many*, aby uzyskać większą skuteczność kosztem dodatkowego czasu [16].

Kolejnym ważnym aspektem wersji 26 jest fakt rezygnacji z użycia DFL (ang. *Distribution Focal Loss*) w trakcie regresji ramek, co pozwala zmniejszyć złożoność głowicy

decyzyjnej i usunąć ograniczenia wynikające z dyskretnej reprezentacji odległości. Do procesu uczenia wprowadzono mechanizmy Progressive Loss oraz STAL (ang. *Small-Target-Aware Label Assignment*), poprawiający przypisanie przykładów pozytywnych dla małych obiektów. W procesie optymalizacji wykorzystano optymalizator MuSGD [16].



Rysunek 2.2: Uproszczony schemat architektury detektora YOLO26 [16].

Na rysunku 2.2 przedstawiono uproszczony schemat współczesnej architektury modelu YOLO26. Możemy zauważyć znaczny rozwój względem pierwszej wersji modelu i wyraźne odejście od przekształcania wejścia w pojedynczą siatkę predykcji. Zastąpiono ją hierarchicznym przetwarzaniem cech, ich wieloskalowym połączeniem i dostosowaną głowicą decyzyjną. Pomimo tych zmian, zachowano główną ideę rodziny, czyli detekcję przez jeden model, bez osobnego etapu generowania propozycji regionów.

2.5.2 Warianty ze względu na realizowane zadanie

Dzisiejsza implementacja rodziny YOLO przez Ultralytics nie ogranicza się jedynie do klasycznej detekcji, a stara się dostarczać wielu modeli obsługujących różne zadania. Głównym kierunkiem jest wciąż model klasycznej detekcji, ale jego architektura może zostać rozszerzona o dodatkowe głowice przeznaczone do obsługi innych zadań. Obecnie dostępne modele mogą obsługiwać zadania detekcji, klasyfikacji, segmentacji instancji, segmentacji semantycznej, estymacji głębi, określania pozy, a od wersji YOLO26 również detekcji z obroconymi ramkami [39], poszczególne zadania zostały wyrażone w formie tabeli 2.3.

Ze względu na specyfikę systemu, spośród dostępnych wariantów YOLO26 zdecydowano się na wykorzystanie podstawowego, zajmującego się detekcją obiektów na obrazie. Wynik w postaci prostokątnej ramki obiektu powinien być wystarczający, aby dostarczyć wystarczającą ilość informacji o obiekcie. Segmentacja umożliwiłaby kompletne wyodrębnianie obiektu z tła, ale wymagałoby to przygotowania masek segmentacyjnych dla zbioru

Tabela 2.3: Wybrane zadania obsługiwane przez modele YOLO26

Zadanie	Oznaczenie modelu	Wynik działania
Detekcja obiektów	yolo26*.pt	Klasy obiektów, wartości pewności i prostokątne ramki ograniczające
Segmentacja instancji	yolo26*-seg.pt	Klasy, ramki ograniczające i osobne maski pikselowe obiektów
Segmentacja semantyczna	yolo26*-sem.pt	Przypisanie klasy do każdego piksela obrazu
Estymacja głębi	yolo26*-depth.pt	Mapa przewidywanej głębi obrazu
Klasyfikacja obrazu	yolo26*-cls.pt	Klasa przypisana do całego obrazu
Estymacja pozy	yolo26*-pose.pt	Położenie punktów charakterystycznych obiektu lub sylwetki
Detekcja z ramkami obróconymi	yolo26*-obb.pt	Klasy i ramki ograniczające rozszerzone o orientację obiektu

uczącego, a także wydłużyłaby czas detekcji. Ciekawym kierunkiem rozwoju może być wykorzystanie wariantu z ramkami obróconymi, wprowadzonym w najnowszej wersji YOLO, aby ograniczyć udział tła w ramce detekcji.

2.5.3 Warianty ze względu na skalę modelu

Każdy wariant YOLO26 jest dostępny w kilku skalach różniących się od siebie liczbą parametrów oraz kosztem obliczeniowym. Wybór odpowiedniego wariantu powinien być oparty na wymaganiach systemu oraz ograniczeniach zasobów sprzętowych urządzenia końcowego. Autorzy modelu detekcyjnego przewidzieli pięć wariantów: **n** (*nano*), **s** (*small*), **m** (*medium*), **l** (*large*) oraz **x** (*extra large*). Każdy kolejny wariant pozwala wykorzystać większą liczbę parametrów prowadząc do lepszej precyzji i dokładności detekcji, ale jednocześnie zwiększa się wymagany koszt obliczeniowy zwiększając czas odpowiedzi oraz wymagania sprzętowe [39].

Tabela 2.4: Porównanie wariantów detekcyjnych modelu YOLO26 na zbiorze COCO

Wariant	mAP50-95	Parametry [mln]	GFLOPs	T4 TensorRT [ms]
YOLO26n	40,9	2,4	5,4	1,7
YOLO26s	48,6	9,5	20,7	2,5
YOLO26m	53,1	20,4	68,2	4,7
YOLO26l	55,0	24,8	86,4	6,2
YOLO26x	57,5	55,7	193,9	11,8

Wyniki, które zostały przedstawione w tabeli 2.4 pokazują kompromis, typowy dla tej rodziny modeli. Wzrost rozmiaru modelu skorelowany jest ze wzrostem wartości mAP_{50-95} , liczbą parametrów, liczbą wykonywanych operacji, oraz czasem wykonania. Największy wzrost skuteczności obserwujemy przy przejściu z modelu YOLO26n do YOLO26s, wynoszącym 7,7 punktu mAP pierwszego z nich, a dalsze zwiększanie rozmiaru przynosi coraz mniejsze przyrosty względem rosnącego kosztu.

Wartości czasowe służą porównaniu wariantów w jednakowych warunkach i nie mogą być utożsamiane z wartością pewną czasu wykonania. Ten zależy od sprzętu, na którym uruchamiany jest model, oraz innych czynników wynikających z implementacji danego systemu.

2.5.4 Dobór wariantów modelu

Do dalszej analizy przyjęto najnowszą wersję modeli rozwijanych przez Ultralytics – YOLO26. Pozwala to oprzeć system na aktualnej architekturze, która unowocześnia wszelkie rozwiązania w kolejnych wersjach modeli rodziny YOLO. Porównanie z wcześniejszymi wersjami pozwala stwierdzić, że najnowsza wersja wprowadza znaczące poprawki i unowocześnienia: rozwój podejścia do detekcji end-to-end oraz ograniczenie liczby operacji.

Ze względu na wymaganie działania modelu w RT oraz założenie wykorzystania systemu w środowisku szkolnym, dalszą analizę należy skupić na najmniejszych wariantach modelu: **n** i **s**. Wynika to z konieczności uwzględnienia ograniczeń i heterogeniczności infrastruktury sprzętowej w polskich placówkach oświatowych. Zieliński w badaniu z 2023 roku dotyczącym cyfryzacji polskich szkół wskazuje na problem przestarzałego sprzętu komputerowego, często mającego ponad 5 lat użytkowania [50].

Wariant **n** charakteryzuje się najmniejszą liczbą parametrów i najmniejszym kosztem wykonywanych obliczeń, przez co jest idealny dla jednostek o niskich zasobach i ograniczonych możliwościach. Kolejny wariant **s** stanowi kolejny poziom złożoności sieci neuronowej, co wskazuje na prawdopodobieństwo lepszych wyników detekcji. Działanie tych modeli w systemie należy przetestować na rzeczywistych urządzeniach i dopasować do warunków rzeczywistego środowiska.

2.6 Wykorzystanie danych RGB, głębi i LiDAR w rozpoznawaniu obiektów

Tu mogę opisać, dlaczego LiDAR był rozważany, ale w praktyce ma ograniczenia przy małych obiektach i małych odległościach.

2.7 Metryki oceny detekcji obiektów

Ocena modeli detekcji obiektów wymaga zastosowania odpowiednich metryk, które pozwalają na porównanie uzyskanych wyników. Metryki te powinny uwzględniać zarówno poprawność klasyfikacji, jak i dokładność lokalizacji obiektu na obrazie. Przeciwnie do klasyfikacji obrazów, przy detekcji nie wystarczy, aby model poprawnie rozpoznał klasę obiektu. Równie istotne jest, aby poprawnie wyznaczył ramkę ograniczającą wokół wykrytego obiektu. Jedną z najczęściej stosowanych metryk jest IoU (ang. *Intersection over Union*), określająca stopień pokrycia między ramką obiektu przewidywaną przez model a ramką rzeczywistą [26]. Wartość IoU obliczana jest jako stosunek pola części wspólnej obu ramek do pola ich części łącznej:

$$IoU = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}, \quad (2.3)$$

gdzie B_p jest ramką przewidywaną przez model, a B_{gt} jest ramką referencyjną (ang. *ground truth*). Im większa wartość IoU, tym lepsze dopasowanie ramki przewidywanej do ramki rzeczywistej. W praktyce najczęściej stosuje się progi, np. $IoU \geq 0.5$, aby zdecydować, czy dana detekcja może być uznana za poprawną. Próg ten został wykorzystany w wielu klasycznych procedurach oceny detekcji, takich jak PASCAL VOC [11], i jest nadal powszechnie stosowany w wielu benchmarkach. Na podstawie wartości IoU oraz zgodności klasy obiektu możliwe jest przypisanie wyników detekcji do jednej z kategorii: prawdziwych pozytywów (TP, ang. *true positive*), fałszywych pozytywów (FP, ang. *false positive*) i fałszywych negatywów (FN, ang. *false negative*). Detekcja jest traktowana jako TP, jeżeli model poprawnie rozpoznał klasę obiektu, a przewidywana ramka spełnia zadany próg IoU względem ramki referencyjnej. FP to detekcja błędna w której model przewidział klasę obiektu, ale nie spełnił progu IoU lub przewidziana klasa była błędna. FN natomiast to sytuacja, w której model nie wykrył obiektu, który rzeczywiście był na obrazie.

Jedną z podstawowych metryk oceny jest precyzja (ang. *precision*), określająca udział poprawnych detekcji modelu wśród wszystkich detekcji, jakie zwrócił:

$$Precision = \frac{TP}{TP + FP}. \quad (2.4)$$

Wysoka precyzja oznacza, że model generuje niewiele fałszywych pozytywów. Drugą, powiązaną z nią metryką, jest czułość (ang. *recall*), określająca, jaki jest udział poprawnych detekcji wśród wszystkich rzeczywistych obiektów:

$$Recall = \frac{TP}{TP + FN}. \quad (2.5)$$

Obie metryki pozostają ze sobą w relacji kompromisu, ponieważ zwiększenie progu pewności detekcji może ograniczyć liczbę fałszywych pozytywów, ale jednocześnie prowadzić do pominięcia części rzeczywistych obiektów. Z kolei obniżenie tego progu może zwiększyć liczbę detekcji, ale równocześnie powodować wzrost liczby fałszywych detekcji. Z tego względu, przy analizie detektorów często łączy się je w jedną metrykę – krzywą PR (precyzja-czułość, ang. *precision-recall curve*), przedstawiającą zależność pomiędzy nimi dla różnych wartości progu decyzyjnego. Na jej podstawie wyznacza się metrykę AP (ang. *Average Precision*), odpowiadającą polu pod krzywą PR.

W przypadku detekcji wieloklasowej używa się średniej wartości AP obliczonej dla wszystkich klas, określanej jako mAP (ang. *mean Average Precision*):

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i, \quad (2.6)$$

gdzie N jest liczbą klas, a AP_i jest wartością AP dla i -tej klasy. W praktyce najczęściej wykorzystuje się dwie wersje tej metryki: mAP_{50} , gdzie detekcję uznaje się za poprawną przy progu $IoU \geq 0.5$, oraz mAP_{50-95} , gdzie wynik jest uśredniany dla progów IoU z zakresu od 0.5 do 0.95. Pierwsza z nich jest bardziej liberalna i pozwala określić ogólną zdolność modelu do wykrycia obiektu. Druga natomiast jest bardziej rygorystyczna, ponieważ wymaga dokładniejszego dopasowania ramek ograniczających, dzięki czemu lepiej pokazuje zarówno skuteczność klasyfikacji, jak i lokalizacji obiektu. Uśrednianie wyników dla wielu progów zostało szeroko zastosowane m. in. w metodyce oceny detekcji zbioru Microsoft COCO [19] i jest obecnie standardem w wielu benchmarkach detekcji obiektów.

Dodatkowo, w pracy wykorzystano metrykę $F1$ -score, będącą średnią harmoniczną precyzji i czułości:

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}, \quad (2.7)$$

gdzie P jest precyzją, a R jest czułością. Metryka ta umożliwia przedstawienie kompromisu pomiędzy liczbą fałszywych detekcji a liczbą pominiętych obiektów. Może być szczególnie przydatna, gdy sama precyzja lub czułość nie dają pełnego obrazu jakości modelu. W niniejszej pracy metrykę $F1$ -score wykorzystano jako metrykę pomocniczą względem podstawowych metryk mAP_{50} i mAP_{50-95} , aby lepiej zrozumieć kompromis pomiędzy precyzją a czułością w kontekście detekcji obiektów dotykanych przez użytkownika.

Oprócz wartości liczbowych wykorzystano również macierze pomyłek (ang. *confusion matrix*), pozwalające przeanalizować, które klasy obiektów są najczęściej mylone przez model. Są one przedstawiane w formie tabel, w których jedna oś odpowiada rzeczywistym klasom, a druga klasom przewidywanym przez model. Dla dobrze działającego modelu większość wartości powinna znajdować się na przekątnej macierzy, co wskazuje na

poprawną klasyfikację obiektów. Macierze pomyłek są szczególnie istotne przy rozpoznawaniu brył geometrycznych o podobnych kształtach, np. sześcianu i prostopadłościanu, ponieważ sama wartość mAP nie wskazuje bezpośrednio, pomiędzy którymi klasami model popełnia najwięcej błędów. Analiza macierzy pomyłek może również pomóc w identyfikacji problemów z danymi treningowymi, np. niedostatecznej liczby przykładów dla wybranych klas lub błędów w oznaczeniach.

2.8 Podsumowanie analizy tematu

Najważniejsze: z literatury wynika, że detekcja obiektów w czasie rzeczywistym jest sensownym kierunkiem, ale Twoja nisza to rozpoznawanie konkretnych dotykanych brył 3D w kontrolowanym scenariuszu.

DRAFT

Rozdział 3

Koncepcja systemu rozpoznawania dotykanych obiektów 3D

To odpowiada wymaganej części „przedmiot pracy”: rozwiązanie zaproponowane przez dyplomanta, analiza teoretyczna, uzasadnienie metod, algorytmów i narzędzi.

3.1 Założenia funkcjonalne i нефunkcjonalne

Funkcjonalne: akwizycja obrazu, przesyłanie klatek, detekcja obiektu, informacja zwrotna. Niefunkcjonalne: możliwie niskie opóźnienie, prostota użycia, mobilność, powtarzalność badań.

3.2 Architektura systemu

iPhone → przesyłanie obrazu/WebSocket → serwer Python → model YOLO → wynik → feedback.

3.3 Dobór klas rozpoznawanych obiektów

Cube, sphere, cylinder, cuboid, tetrahedron, cone. Tu warto uzasadnić, że są to podstawowe bryły geometryczne, możliwe do wydruku 3D i łatwe do kontrolowanego oznaczania.

3.4 Uzasadnienie wyboru metody detekcji

Dlaczego YOLO: szybkość, popularność, gotowe narzędzia treningowe, możliwość porównania wariantów n/s, metryki.

3.5 Przepływ danych w systemie

Od obrazu z telefonu, przez detekcję, po komunikat dla użytkownika.

3.6 Ograniczenia przyjętej koncepcji

Na przykład: kontrolowany zestaw obiektów, zależność od oświetlenia, ograniczony dataset, problem z generalizacją na inne dłonie/osoby/tła.

DRAFT

Rozdział 4

Implementacja systemu

Tu opisuję konkretną robotę inżynierską. Nie za dużo kodu w głównym tekście — raczej architektura, moduły, decyzje. Opis interfejsów aplikacji i środowisk projektowania lepiej dawać jako załącznik, bo wymagania sugerują, że takie rzeczy powinny iść do załącznika.

4.1 Środowisko programistyczne i wykorzystane narzędzia

Python, PyTorch/Ultralytics, OpenCV, Swift/iOS, ARKit, WebSocket, CVAT/Robo-flow/HF — tylko te, które finalnie zostaną.

4.2 Moduł akwizycji danych obrazowych

Jak pozyskuję obraz z telefonu / nagrań / streamu.

4.3 Moduł komunikacji klient-serwer

WebSocket, przesyłanie ramek, format danych, obsługa połączenia.

4.4 Moduł detekcji obiektów

Ładowanie modelu, inferencja, wynik detekcji, progi confidence.

4.5 Moduł informacji zwrotnej dla użytkownika

Komunikaty audio / tekstowe / docelowe zachowanie aplikacji.

4.6 Organizacja kodu i plików projektu

Krótko, bez robienia instrukcji obsługi. Dokładna dokumentacja może iść do załącznika albo plików dodatkowych.

DRAFT

Rozdział 5

Zbiór danych i metodyka badań

To jest bardzo ważny rozdział, bo wymagania kładą nacisk na metodykę, dane, powtarzalność i naukową analizę wyników. Badania powinny być przeprowadzone „tak jak jest przyjęte w ujęciu naukowym”, np. z zapewnieniem powtarzalności testów.

5.1 Charakterystyka zbioru danych

Liczba klas, liczba obrazów, rozdzielczość, źródło danych, przykładowe obrazy.

5.2 Proces pozyskiwania i oznaczania danych

Nagrania, ekstrakcja klatek, anotacje, negatywne próbki, puste etykiety.

5.3 Podział danych na zbiory treningowy, walidacyjny i testowy

Train/val/test. Tu trzeba bardzo jasno wyjaśnić, jak dzielić dane, żeby recenzent nie zarzucił przecieku danych.

5.4 Warianty danych wejściowych

Kolor, grayscale, ewentualne filtry/normalizacja. To będzie dobre miejsce na porównanie „kolor vs grayscale”.

5.5 Konfiguracja treningu modeli

Modele, rozmiar obrazu, batch, liczba epok, patience, pretrained/from scratch, sprzęt GPU.

5.6 Metryki oceny

Precision, recall, mAP50, mAP50-95, macierz pomyłek, czas inferencji/FPS, zużycie pamięci.

Do oceny modeli wykorzystano metryki zwracane przez narzędzie Ultralytics YOLO: precision, recall, mAP50 oraz mAP50-95. Dodatkowo na podstawie wartości precision i recall obliczono F1-score. Wyniki zapisywano w pliku results.csv, natomiast przebiegi metryk oraz macierze pomyłek analizowano na podstawie plików generowanych po zakończeniu treningu i walidacji. [38]

5.7 Procedura eksperymentalna

Czyli dokładnie: jakie modele trenowano, na jakich danych, jakie wyniki zbierano, jak porównywano.

Rozdział 6

Wyniki badań i ich analiza

To powinien być osobny rozdział, nie mieszany z metodyką. Wymagania mówią, że badania mają zawierać prezentację wyników, opracowanie i poszerzoną dyskusję wyników oraz wnioski.

6.1 Wyniki uczenia modeli YOLO

Tabele z najlepszymi wynikami: model, imgsz, precision, recall, mAP50, mAP50-95, f1, czas, pamięć.

6.2 Porównanie wariantów modeli

YOLO n vs s, ewentualnie inne warianty.

6.3 Wpływ reprezentacji obrazu na skuteczność detekcji

Kolor vs grayscale. To może być jeden z ciekawszych fragmentów pracy.

6.4 Analiza błędów klasyfikacji i detekcji

Macierze pomyłek, najczęściej mylone klasy, np. cube/sphere/cuboid itd.

6.5 Ocena działania systemu w scenariuszu użytkowym

Nie tylko mAP, ale też: czy system nadaje się do użycia, jakie ma opóźnienia, co działa, co nie działa.

6.6 Dyskusja ograniczeń wyników

Mały/średni dataset, kontrolowane warunki, jedna osoba w części danych, zależność od tła/oświetlenia, możliwe przeuczenie, generalizacja.

DRAFT

Rozdział 7

Podsumowanie i wnioski końcowe

7.1 Podsumowanie wykonanych prac

Co zostało zrobione: dataset, modele, system, eksperymenty.

7.2 Ocena realizacji celu pracy

Wprost: czy cel został osiągnięty, w jakim zakresie, z jakimi ograniczeniami.

7.3 Najważniejsze wnioski

Np. detekcja obiektów jest skuteczna, ale ma ograniczenia, np. w przypadku małych obiektów, podobnych kształtów, zależności od tła i oświetlenia.

7.4 Możliwe kierunki rozwoju

Większy dataset, więcej osób, więcej obiektów, integracja LiDAR/RGB-D, inference lokalnie na iPhone, lepszy feedback audio, testy z użytkownikami.

DRAFT

Bibliografia

- [1] Mirza Samad Ahmed Baig, Syeda Anshrah Gillani, Shahid Munir Shah, Mahmoud Aljawarneh, Abdul Akbar Khan i Muhammad Hamzah Siddiqui. *AI-based Wearable Vision Assistance System for the Visually Impaired: Integrating Real-Time Object Recognition and Contextual Understanding Using Large Vision-Language Models*. 2024. arXiv: 2412.20059 [cs.CV]. URL: <https://arxiv.org/abs/2412.20059>.
- [2] Fabiola Cando-Guanoluisa, Lesly Claudio, Dylan Cevallo, Carlos Colcha, Silvia Taípe i Grecia Pilatas. „Visually Impaired Students’ and Their Teacher’s Perceptions of the English Teaching and Learning Process”. W: *Mertesol Journal* 46 (sty. 2023), s. 1–14. DOI: 10.61871/mj.v46n4-3.
- [3] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov i Sergey Zagoruyko. *End-to-End Object Detection with Transformers*. 2020. arXiv: 2005.12872 [cs.CV]. URL: <https://arxiv.org/abs/2005.12872>.
- [4] João Francisco Staffa Costa, V. M. R. Lima i M. S. Biembengut. „Spatial perception of a blind person: interactions and experiences”. W: *Brazilian Journal of Science Teaching and Technology, Ponta Grossa* 16 (2023), s. 1–20. DOI: 10.3895/rbect.v16n1.13944.
- [5] N. Dalal i B. Triggs. „Histograms of oriented gradients for human detection”. W: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’05)*. T. 1. 2005, 886–893 vol. 1. DOI: 10.1109/CVPR.2005.177.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit i Neil Houlsby. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
- [7] Ewa Dzierzgowska, Michał Maćkowski, Mateusz Kawulok, Piotr Brzoza, Stella Maćkowska i Dominik Spinczyk. „Alternative audio-graphic method for presenting structural information in mathematical graphs designed for low-vision users”. W: *Scientific Reports* 15.1 (2025), s. 21972. ISSN: 2045-2322. DOI: 10.1038/s41598-025-07710-2. URL: <https://doi.org/10.1038/s41598-025-07710-2>.

- [8] ETSI, CEN and CENELEC. *EN 301 549 V3.2.1: Accessibility Requirements for ICT Products and Services*. European Harmonised Standard. Mar. 2021. URL: https://www.etsi.org/deliver/etsi_en/301500_301599/301549/03.02.01_60/en_301549v030201p.pdf (term. wiz. 11.05.2026).
- [9] European Parliament and Council of the European Union. *Directive (EU) 2016/2102 of the European Parliament and of the Council of 26 October 2016 on the accessibility of the websites and mobile applications of public sector bodies*. Official Journal of the European Union, L 327, 2 December 2016, pp. 1–15. 2016. URL: <https://eur-lex.europa.eu/eli/dir/2016/2102/oj> (term. wiz. 11.05.2026).
- [10] European Parliament and Council of the European Union. *Directive (EU) 2019/882 of the European Parliament and of the Council of 17 April 2019 on the Accessibility Requirements for Products and Services*. Official Journal of the European Union, L 151, 7 June 2019, pp. 70–115. 2019. URL: <https://eur-lex.europa.eu/eli/dir/2019/882/oj> (term. wiz. 11.05.2026).
- [11] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn i Andrew Zisserman. „The PASCAL Visual Object Classes (VOC) Challenge.” W: *International Journal of Computer Vision* 88.2 (2010), s. 303–338. DOI: 10.1007/s11263-009-0275-4.
- [12] Lourdes Figueiras i Abraham Arcavi. „Learning to See: The Viewpoint of the Blind”. W: *Selected Regular Lectures from the 12th International Congress on Mathematical Education*. Red. Sung Je Cho. Cham: Springer International Publishing, 2015, s. 175–186. ISBN: 978-3-319-17187-6. DOI: 10.1007/978-3-319-17187-6_10.
- [13] Maşide Güler i Mustafa Ürey. „Are we aware of the needs of students with visual impairment? A study on challenges and instructional needs in science lessons”. W: *Journal of Pedagogical Sociology and Psychology* 7 (2 2025), s. 53–70. DOI: 10.33902/jpsp.202529253.
- [14] Kaiming He, Georgia Gkioxari, Piotr Dollár i Ross Girshick. *Mask R-CNN*. 2018. arXiv: 1703.06870 [cs.CV]. URL: <https://arxiv.org/abs/1703.06870>.
- [15] Intel Corporation. *New Intel-Created System Offers Professor Stephen Hawking Ability to Better Communicate with the World*. 2 grud. 2014. URL: <https://www.intel.com/news-events/press-releases/detail/380/new-intel-created-system-offers-professor-stephen-hawking> (term. wiz. 10.05.2026).
- [16] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon i Muhammet Kalfaoglu. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. Czer. 2026. DOI: 10.48550/arXiv.2606.03748.

- [17] Franklin Mingzhe Li, Lotus Zhang, Maryam Bandukda, Abigale Stangl, Kristen Shinohara, Leah Findlater i Patrick Carrington. „Understanding Visual Arts Experiences of Blind People”. W: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. CHI '23. Hamburg, Germany: Association for Computing Machinery, 2023. ISBN: 9781450394215. DOI: 10.1145/3544548.3580941.
- [18] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He i Piotr Dollár. *Focal Loss for Dense Object Detection*. 2018. arXiv: 1708.02002 [cs.CV]. URL: <https://arxiv.org/abs/1708.02002>.
- [19] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár i C. Lawrence Zitnick. „Microsoft COCO: Common Objects in Context”. W: *Computer Vision – ECCV 2014*. T. 8693. Lecture Notes in Computer Science. Springer, 2014, s. 740–755. DOI: 10.1007/978-3-319-10602-1_48.
- [20] John G. Linvill. „Reading Aid for the Blind”. Pat. US 3,229,387 (United States). 18 sty. 1966. URL: <https://patents.google.com/patent/US3229387A/en>.
- [21] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu i Alexander C. Berg. „SSD: Single Shot MultiBox Detector”. W: *Computer Vision – ECCV 2016*. Springer International Publishing, 2016, s. 21–37. ISBN: 9783319464480. DOI: 10.1007/978-3-319-46448-0_2. URL: http://dx.doi.org/10.1007/978-3-319-46448-0_2.
- [22] David G. Lowe. „Distinctive Image Features from Scale-Invariant Keypoints”. W: *International Journal of Computer Vision* 60.2 (2004), s. 91–110. ISSN: 1573-1405. DOI: 10.1023/B:VISI.0000029664.99615.94. URL: <https://doi.org/10.1023/B:VISI.0000029664.99615.94>.
- [23] R. G. Maling i D. C. Clarkson. „Electronic controls for the tetraplegic (possum) (patient operated selector mechanisms—P.O. S.M.)” W: *Spinal Cord* 1.3 (1963), s. 161–174. ISSN: 1476-5624. DOI: 10.1038/sc.1963.15. URL: <https://doi.org/10.1038/sc.1963.15>.
- [24] MarketsandMarkets. *Wearable Technology Market Size, Share & Trends*. 1 sty. 2026. URL: <https://www.marketsandmarkets.com/Market-Reports/wearable-electronics-market-983.html> (term. wiz. 10.05.2026).
- [25] Mara Mills. „Hearing Aids and the History of Electronics Miniaturization”. W: *IEEE Annals of the History of Computing* 33.2 (2011), s. 24–45. DOI: 10.1109/MAHC.2011.43.
- [26] Rafael Padilla, Wesley L. Passos, Thadeu L. B. Dias, Sergio L. Netto i Eduardo A. B. da Silva. „A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit”. W: *Electronics* 10.3 (2021). ISSN: 2079-9292. DOI: 10.3390/electronics10030279.

- [27] Kedar Potdar, Chinmay Pai i Sukrut Akolkar. „A Convolutional Neural Network based Live Object Recognition System as Blind Aid”. W: (list. 2018). DOI: 10.48550/arXiv.1811.10399.
- [28] Abhinav Pratap, Sushant Kumar i Suchinton Chakravarty. *Adaptive Object Detection for Indoor Navigation Assistance: A Performance Evaluation of Real-Time Algorithms*. 2025. arXiv: 2501.18444 [cs.CV]. URL: <https://arxiv.org/abs/2501.18444>.
- [29] Joseph Redmon, Santosh Divvala, Ross Girshick i Ali Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: 1506.02640 [cs.CV]. URL: <https://arxiv.org/abs/1506.02640>.
- [30] Joseph Redmon i Ali Farhadi. *YOLOv3: An Incremental Improvement*. 2018. arXiv: 1804.02767 [cs.CV]. URL: <https://arxiv.org/abs/1804.02767>.
- [31] Shaoqing Ren, Kaiming He, Ross Girshick i Jian Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. 2016. arXiv: 1506.01497 [cs.CV]. URL: <https://arxiv.org/abs/1506.01497>.
- [32] Rzeczpospolita Polska. *Ustawa z dnia 26 kwietnia 2024 r. o zapewnianiu spełniania wymagań dostępności niektórych produktów i usług przez podmioty gospodarcze*. Dz.U. z 2024 r. poz. 731. 2024. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20240000731> (term. wiz. 11.05.2026).
- [33] Rzeczpospolita Polska. *Ustawa z dnia 4 kwietnia 2019 r. o dostępności cyfrowej stron internetowych i aplikacji mobilnych podmiotów publicznych*. Dz.U. z 2019 r. poz. 848; tekst jednolity: Dz.U. z 2023 r. poz. 1440, z późn. zm. 2019. URL: <https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20190000848> (term. wiz. 11.05.2026).
- [34] Pierre Sermanet, David Eigen, Xiang Zhang, Michael Mathieu, Rob Fergus i Yann LeCun. *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks*. 2014. arXiv: 1312.6229 [cs.CV]. URL: <https://arxiv.org/abs/1312.6229>.
- [35] Sibi C. Sethuraman, Gaurav R. Tadkapally, Saraju P. Mohanty, Gautam Galada i Anitha Subramanian. *MagicEye: An Intelligent Wearable Towards Independent Living of Visually Impaired*. 2023. arXiv: 2303.13863 [cs.HC]. URL: <https://arxiv.org/abs/2303.13863>.
- [36] Dandan Shan, Jiaqi Geng, Michelle Shu i David F. Fouhey. „Understanding Human Hands in Contact at Internet Scale”. W: 2020. arXiv: 2006.06669 [cs.CV]. URL: <https://arxiv.org/abs/2006.06669>.

- [37] Bugra Tekin, Federica Bogo i Marc Pollefeys. „H+O: Unified Egocentric Recognition of 3D Hand-Object Poses and Interactions”. W: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, s. 4506–4515. DOI: 10.1109/CVPR.2019.00464.
- [38] Ultralytics. *Performance Metrics Deep Dive*. YOLO Performance Metrics documentation. 12 list. 2023. URL: <https://docs.ultralytics.com/guides/yolo-performance-metrics/> (term. wiz. 17.12.2025).
- [39] Ultralytics. *Ultralytics YOLO26*. 25 wrz. 2025. URL: <https://docs.ultralytics.com/models/yolo26> (term. wiz. 04.11.2025).
- [40] Ultralytics. *YOLO Architecture Explained: From YOLOv3 to YOLO26*. 6 lip. 2026. URL: <https://docs.ultralytics.com/guides/yolo-architecture> (term. wiz. 10.07.2026).
- [41] United Nations. *Convention on the Rights of Persons with Disabilities*. United Nations General Assembly Resolution A/RES/61/106. 2006. URL: <https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities> (term. wiz. 11.05.2026).
- [42] Gregg Vanderheiden. „A journey through early augmentative communication and computer access”. W: *Journal of rehabilitation research and development* 39 (sty. 2003), s. 39–54.
- [43] P. Viola i M. Jones. „Rapid object detection using a boosted cascade of simple features”. W: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*. T. 1. 2001, s. I–I. DOI: 10.1109/CVPR.2001.990517.
- [44] Wei Wang, Bin Jing, Xiaoru Yu, Yan Sun, Liping Yang i Chunliang Wang. „YOLO-OD: Obstacle Detection for Visually Impaired Navigation Assistance”. W: *Sensors* 24.23 (2024). ISSN: 1424-8220. DOI: 10.3390/s24237621. URL: <https://www.mdpi.com/1424-8220/24/23/7621>.
- [45] World Wide Web Consortium. *Web Content Accessibility Guidelines 1.0*. W3C Recommendation. Maj 1999. URL: <https://www.w3.org/TR/WAI-WEBCONTENT/> (term. wiz. 11.05.2026).
- [46] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. 2024. URL: <https://www.w3.org/TR/WCAG22/> (term. wiz. 11.05.2026).
- [47] Jingxi Xu, Han Lin, Shuran Song i Matei Ciocarlie. *TANDEM3D: Active Tactile Exploration for 3D Object Recognition*. 2023. DOI: 10.48550/arXiv.2209.08772. arXiv: 2209.08772 [cs.CV]. URL: <https://arxiv.org/abs/2209.08772>.

- [48] Chenyangguang Zhang, Guanlong Jiao, Yan Di, Gu Wang, Ziqin Huang, Ruida Zhang, Fabian Manhardt, Bowen Fu, Federico Tombari i Xiangyang Ji. „MOHO: Learning Single-view Hand-held Object Reconstruction with Multi-view Occlusion-Aware Supervision”. W: 2024. arXiv: 2310.11696 [cs.CV]. URL: <https://arxiv.org/abs/2310.11696>.
- [49] Xuefeng ZHANG, Shaojie ZHANG, Xin CHEN i Jinhua ZHANG. „A tactile glove for object recognition based on palmar pressure and joint bending strain sensing”. W: *Journal of Measurement Science and Instrumentation* 16.2 (2025), s. 173–185. DOI: 10.62756/jmsi.1674-8042.2025017. URL: <https://www.sciopen.com/article/10.62756/jmsi.1674-8042.2025017>.
- [50] Robert Zieliński. „Cyfrowa edukacja dziś:między szansą a wyzwaniem – studium porównawcze”. W: *EBIS* 2.80 (2023), s. 234–267. ISSN: 1643-8779. DOI: 10.24131/3247.230212.
- [51] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo i Jieping Ye. „Object Detection in 20 Years: A Survey”. W: *Proceedings of the IEEE* 111.3 (2023), s. 257–276. DOI: 10.1109/JPROC.2023.3238524.

Dodatki

DRAFT

Dokumentacja techniczna

DRAFT

DRAFT

Spis skrótów i symboli

- IoU Intersection over Union – miara stopnia pokrycia ramki przewidywanej przez model z ramką referencyjną,
- TP True Positive – prawdziwy pozytywny, poprawnie wykryty obiekt,
- FP False Positive – fałszywy pozytywny, błędna detekcja zwrócona przez model,
- FN False Negative – fałszywy negatywny, obiekt rzeczywisty niewykryty przez model,
- PR Precision-Recall – zależność między precyzją a czułością,
- AP Average Precision – średnia precyzja wyznaczana na podstawie krzywej precision-recall,
- mAP mean Average Precision – średnia wartość AP obliczana dla wszystkich klas,
- F1 F1-score – średnia harmoniczna precyzji i czułości,
- VOC Visual Object Classes – zbiór nazw benchmarku PASCAL VOC wykorzystywanego w ocenie detekcji obiektów,
- COCO Common Objects in Context – nazwa zbioru i benchmarku Microsoft COCO,
- W3C World Wide Web Consortium – organizacja zajmująca się standaryzacją technologii internetowych,
- WCAG Web Content Accessibility Guidelines – wytyczne dotyczące dostępności treści internetowych,
- CNN Convolutional Neural Network – konwolucyjna sieć neuronowa,
- HOG Histogram of Oriented Gradients – histogramy gradientów kierunkowych,
- SIFT Scale-Invariant Feature Transform – transformacja cech niezależnych,
- RPN Region Proposal Network – sieć generująca propozycje regionów,
- YOLO You Only Look Once – rodzina modeli detekcji jednoetapowej obiektów w czasie rzeczywistym,

DETR DETection TRansformer – model detekcji obiektów wykorzystujący architekturę transformera,

NMS Non-Maximum Suppression – algorytm usuwania nakładających się ramek detekcji,

RT Real-Time – czas rzeczywisty,

SSD Single Shot MultiBox Detector – model detekcji jednoetapowej obiektów w czasie rzeczywistym,

FPS Frames Per Second – liczba kadrów na sekundę,

DFL Distribution Focal Loss – funkcja straty wykorzystywana w modelach detekcji obiektów,

Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- zbiory danych użyte w eksperymentach,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

DRAFT

Spis rysunków

2.1	Uproszczony schemat architektury pierwszego modelu YOLO [29].	18
2.2	Uproszczony schemat architektury detektora YOLO26 [16].	20

DRAFT

Spis tabel

2.1	Porównanie wybranych metod detekcji obiektów	16
2.2	Wybrane etapy rozwoju architektury modeli YOLO. Opracowanie własne na podstawie [16, 29, 30, 40].	19
2.3	Wybrane zadania obsługiwane przez modele YOLO26	21
2.4	Porównanie wariantów detekcyjnych modelu YOLO26 na zbiorze COCO .	21